BÁO CÁO THỰC HÀNH

Thực hành an toàn Mạng máy tính

Lab 03: Mã hóa bất đối xứng RSA

Hàm băm và chữ ký số

GVTH: KS.Nguyễn Ngọc Trưởng

Ngày báo cáo: 03/04/2026

Nhóm 08 – NT101.Q21.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24520115	Nguyễn Quang Thế Anh	Tất cả	100%

1. Mã hóa khóa công khai RSA

Nhiệm vụ 2.1:

I. Ý tưởng: viết một chương trình để minh họa cách hoạt động của mật mã ngẫu nhiên nếu p, q, e không được cung cấp. Lưu ý rằng cặp khóa phải lớn nhất có thể. Sử dụng các khóa đã tạo để mã hóa/giải mã thông điệp. Thông điệp có thể là số hoặc chuỗi.

1. Xác định khóa công khai PU và khóa riêng PR, nếu:
   - p1 = 11, q1 = 17, e1 = 7 (hệ thập phân)
   - p2 = 2007999387284232211615121 9,

   q2 = 676717145751736242170789,

   e2 = 17 (hệ thập phân)
   - p3 = F7E75FDC469067FFDC4E847C51F452DF,

   q3 = E85CED54AF57E53E092113E62F436F4F,

   e3 = 0D88C3 (hệ thập lục phân)
2. Sử dụng khóa được tạo ra từ p1, q1, e1 để mã hóa và giải mã bản rõ M=5 trong cả hai trường hợp Mã hóa cho tính bảo mật (Encryption for Confidentiality) và Mã hóa cho tính xác thực (Encryption for Authentication).
3. Sử dụng các khóa trên để mã hóa thông điệp sau: The University of Information Technology. Xác định bản mã dưới dạng Base64.
4. Tìm bản rõ tương ứng của mỗi bản mã sau, biết rằng chúng được mã hóa bằng một trong ba khóa đã cho ở trên.
   - raUcesUlOkx/8ZhgodMoo0Uu18sC20yXlQFevSu7W/FDxIy0YRHM yX cHdD9PBvIT2aUft5fCQEGomiVVPv4I
   - C87F570FC4F699CEC24020C6F54221ABAB2CE0C3
   - Z2BUSkJcg0w4XEpgm0JcMExEQmBlVH6dYEpNTHpMHptMQ7Ng T HlgQrNMQ2BKTQ==

- 00101000000101001111111101101110010111011001010111011001
  0110011110111111001111110110100011001111001100001001010 0
  1010100111101 0101001100111011101110111101011011000001 00

II. Thực hiện ý tưởng:

1. Xác định khóa công khai PU và khóa riêng PR
   - Cho p1=11, q1=17, e1=7.
   - Tính n: n=p1×q1=11×17=187.
   - Tính hàm số Euler f(n): f(n)=(p1−1) × (q1−1) =10×16=160.
   - Tìm khóa bí mật d:
     - Cần tìm d sao cho e1×d≡1(mod160). Ta có 7×23=161≡1(mod160), => d=23.

   Kết luận:

   Khóa công khai: PU= (e1, n) = (7,187).

   Khóa riêng tư: PR=(d,n)= (23,187).

2. Mã hóa và giải mã thông điệp M = 5

   Trường hợp Mã hóa cho tính bảo mật (Confidentiality):

   - Mã hóa bằng khóa công khai (PU): $C=M^{e1}(\text{mod}n)$.

     Suy ra: $C=5^{7}(\text{mod}187)$ =78125(mod187) =146.

   - Giải mã bằng khóa riêng (PR): $M=C^{d}(\text{mod}n)$.

     Suy ra: $M=146^{23}(\text{mod}187)$ =5.

   Trường hợp Mã hóa cho tính xác thực (Authentication):

   - Mã hóa bằng khóa riêng (PR): $C=M^{d}(\text{mod}n)$.

     Suy ra: $C=5^{23}(\text{mod}187)$ =180.

   - Giải mã bằng khóa công khai (PU): $M=C^{e1}(\text{mod}n)$.

     Suy ra: $M=180^{7}(\text{mod}187)$ =5.

3. Mô tả và giải thích các công việc làm bên trong code

Link Toàn bộ Source Code:

https://github.com/TheAnh-05-UIT/NT101-AnToanMangMayTinh/tree/main/Lab03

I. Tổng quan Kiến trúc Chương trình

Chương trình được xây dựng trên ngôn ngữ Python, mô phỏng toàn vẹn quá trình sinh khóa, mã hóa và giải mã của hệ mật mã bất đối xứng RSA. Chương trình có khả năng xử lý các số nguyên tố có kích thước lớn và tích hợp cơ chế mã hóa chia khối (Block Cipher) để khắc phục giới hạn về chiều dài thông điệp của thuật toán RSA.

Chương trình được chia thành 4 phần:

1. Phần Toán học cơ sở: Xử lý tính toán liên quan đến số nguyên tố và số học module.
2. Phần Sinh khóa (Key Generation): Tính toán các thông số N,phi(N) và tìm khóa riêng d.
3. Phần Mã hóa & Giải mã: Chuyển đổi dữ liệu (số học và chuỗi ký tự) theo các thuật toán RSA.
4. Phần Thực thi (Main): Gọi các hàm để giải quyết các yêu cầu cụ thể của bài toán.

II. Giải thích chi tiết các Hàm cốt lõi

2.1. Nhóm hàm Toán học (Sinh khóa và tính nghịch đảo)

- Hàm is_prime(n, k) và generate_large_prime(bits):
  Chương trình áp dụng Thuật toán kiểm tra tính nguyên tố xác suất Miller-Rabin. Hàm generate_large_prime sinh ngẫu nhiên một dải bit lẻ và dùng Miller-Rabin để kiểm tra cho đến khi tìm được một số nguyên tố.
- Hàm mod_inverse(e, phi):
  Hàm này dùng để tìm khóa riêng d, thỏa mãn phương trình: e x d≡1(mod phi(n)). Chương trình áp dụng Thuật toán Euclid mở rộng để tìm nghịch đảo module của e.

2.2. Nhóm hàm Sinh khóa RSA

- Hàm rsa_keygen(p, q, e, bits):

Hàm nhận đầu vào là các giá trị cho trước (p,q,e). Nếu đầu vào rỗng, hệ thống sẽ tự động gọi hàm sinh số nguyên tố để tạo p,q lớn nhất (mặc định 1024-bit).

- Tính module public: n = p x q
- Tính hàm Euler: phi(n)=(p−1) x (q−1)
- Hàm trả về Khóa công khai PU={e,n} và Khóa riêng PR={d,n}.

2.3. Nhóm hàm Mã hóa và Giải mã số học

- Hàm encrypt_number và decrypt_number:

  Áp dụng trực tiếp công thức lý thuyết của RSA:

  - Mã hóa: Bản mã C≡M^e (mod n)
  - Giải mã: Bản rõ M≡C^d (mod n)

  Phép toán này được thực hiện bằng hàm pow(base, exp, mod) của Python, cho phép tính lũy thừa module cực kỳ nhanh với các số siêu lớn nhờ thuật toán bình phương và nhân (Square-and-Multiply).

2.4. Cơ chế Mã hóa Chia Khối cho Chuỗi Ký Tự (Block Cipher)

- Hàm rsa_encrypt_string:

  Thuật toán tự động đo kích thước byte của n, cắt bản rõ ban đầu thành các khối có kích thước nhỏ hơn n đúng 1 byte. Mỗi khối được mã hóa độc lập. Sau khi mã hóa, các khối bản mã được ghép lại và mã hóa toàn bộ thành định dạng Base64 để đảm bảo khả năng truyền tải và in không bị lỗi ký tự.

- Hàm rsa_decrypt_blocks:

  Tiếp nhận chuỗi Base64 (hoặc Hex, Binary), thuật toán tiến hành giải mã từng khối độ dài cố định. Sau đó ghép các byte kết quả lại và dùng bảng mã UTF-8 để phục hồi chuỗi văn bản gốc.

III. Mô tả cách thực hiện 4 Yêu cầu bài tập

1. Yêu cầu 1 (Tính khóa): Chương trình gọi hàm rsa_keygen với các biến đầu vào cho sẵn. Đối với Bộ 3, hệ thập lục phân (Hex) được tự động chuyển đổi sang hệ thập phân (Decimal) thông qua hàm int("Hex_String", 16) của Python trước khi tính toán.

2. Yêu cầu 2 (Mã hóa M=5):

   Confidentiality (Tính Bảo mật): Mô phỏng kịch bản gửi tin mật. Tin nhắn được mã hóa bằng Public Key (PU) của người nhận, đảm bảo chỉ người nhận dùng Private Key (PR) của họ mới giải mã được.

   Authentication (Tính Xác thực/Chữ ký số): Mô phỏng kịch bản xác minh danh tính. Tin nhắn được mã hóa bằng Private Key (PR) của người gửi (đóng vai trò như chữ ký). Bất kỳ ai dùng Public Key (PU) của người gửi để giải mã thành công đều có thể xác nhận nguồn gốc tin nhắn.
3. Yêu cầu 3 (Mã hóa Base64): Sử dụng hàm chia khối rsa_encrypt_string kết hợp với Bộ khóa 3 (vì Bộ 3 có N lớn nhất, đủ khả năng xử lý khối dữ liệu an toàn) để biến đổi đoạn văn bản thành chuẩn Base64.
4. Yêu cầu 4 (Dò tìm Khóa giải mã): Chương trình thiết lập một vòng lặp tự động (Brute-force nhẹ). Đối với mỗi bản mã (đã được làm sạch định dạng Base64, Hex, hoặc Nhị phân về dạng Byte), chương trình thử lần lượt dùng 3 Bộ Khóa (Private Keys) để giải mã bằng cả 2 cách: giải mã số nguyên khối đơn và giải mã chia khối. Nếu kết quả trả về là một chuỗi văn bản hợp lệ (có thể in ra theo chuẩn ASCII/UTF-8 - hàm .isprintable()), chương trình sẽ thông báo thành công và in ra bản rõ.

4. Kết quả khi chạy file code python:

```
============================================================
Yêu cầu 1: Xác định khóa công khai PU và khóa riêng PR
============================================================

Bộ 1:
 PU = (7, 187)
 PR = (23, 187)

Bộ 2:
 PU = (17, 135884761403422083943951664696476272266743485417 91)
 PR = (799322125902482846729126218408035801918587659987 3, 135884761403422083943951664696476272266743485417 91)

Bộ 3 (Hex -> Dec):
 PU = (886979, 101776877529005912638346811918779931246783058062684819617574643018368103302097)
 PR = (24212225287904763939160097464943268930139828978795606022583874367720623008491, 101776877529005912638346811918779931246783058062684819617574643018368103302097)

============================================================
Yêu cầu 2: Mã hóa/giải mã bản rõ M = 5 (bộ 1)
============================================================
Bảo mật (Confidentiality):
 - Mã hóa (Dùng PU): 146
 - Giải mã (Dùng PR): 5

Xác thực (Authentication):
 - Mã hóa/Ký (Dùng PR): 180
 - Giải mã/Xác minh (Dùng PU): 5

============================================================
Yêu cầu 3: Mã hóa chuỗi
============================================================
Bản rõ: The University of Information Technology
Bản mã dùng Bộ 3 (Base64): R9n3NredoQsUmSGNXJ5GDnzxJccF6mqXvGjYuWJOl8rBOYtJFgefuZ2UiSDeV9Q4sa5Fk2TnWquyMD6d9vS54Q==

============================================================
Yêu cầu 4: Tìm bản mã của các bản rõ
============================================================

--- Giải mã Bản mã 1 (BASE64) ---
Không thể giải mã thành văn bản hợp lệ bằng các khóa hiện tại.

--- Giải mã Bản mã 2 (HEX) ---
Không thể giải mã thành văn bản hợp lệ bằng các khóa hiện tại.

--- Giải mã Bản mã 3 (BASE64) ---
[Thành công] Dùng Key 1 (Block Cipher) - Bản rõ Text: Vietnam National University - Ho Chi Minh City
```

Kết quả sau khi chạy code.

1. Kết quả của yêu cầu 1:

   Bộ 1:

   PU = (7, 187)

   PR = (23, 187)

   Bộ 2:

   PU = (17, 135884761403422083943951664696476272266743485417 91)

   PR = (799322125902482846729126218408035801918587659987 3, 135884761403422083943951664696476272266743485417 91)

   Bộ 3 (Hex -> Dec):

   PU = (886979, 101776877529005912638346811918779931246783058062684819617574643018368103302097)

   PR = (24212225287904763939160097464943268930139828978795606022583874

4367720623008491,
101776877529005912638346811918779931246783058062684819617574643018368103302097)

2. Kết quả của yêu cầu 2:
   Bảo mật (Confidentiality):
   - Mã hóa (Dùng PU): 146
   - Giải mã (Dùng PR): 5

   Xác thực (Authentication):
   - Mã hóa/Ký (Dùng PR): 180
   - Giải mã/Xác minh (Dùng PU): 5
3. Kết quả của yêu cầu 3:
   Bản rõ: The University of Information Technology
   Bản mã dùng Bộ 3 (Base64):
   R9n3NredoQsUmSGNXJ5GDnzxJccF6mqXvGjYuWJOl8rBOYtJFgefuZ2UiSDeV9Q4sa5Fk2TnWquyMD6d9vS54Q==
4. Kết quả của yêu cầu 4:
   Kết quả giải mã được: Vietnam National University - Ho Chi Minh City

2. Đặc tính của hàm băm

Nhiệm vụ 2.2:

I. Ý tưởng: tìm hiểu về xung đột MD5 và SHA-1 trong nhiệm vụ này bằng cách thực hiện các bài tập sau:

1. Xem xét hai thông điệp (message) dạng HEX như sau:

Message 1

d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f8955ad340609f4b30283e488832571415a085125e8f7cdc99fd91dbdf280373c5bd8823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70

Message 2

d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f8955ad340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd728073c5bd8823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0e99f33420f577ee8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70

Có bao nhiêu byte khác biệt giữa hai thông điệp? Hãy tạo giá trị băm MD5 cho mỗi thông điệp và quan sát xem các giá trị MD5 này có giống nhau hay không và mô tả quan sát trong báo cáo lab.

2. Xem xét hai chương trình thực thi có tên hello and erase: Chạy các chương trình này từ console và quan sát điều gì xảy ra. Hãy tạo giá trị băm MD5 cho các chương trình này và báo cáo quan sát.
3. Tải 2 tệp tin PDF: shattered-1.pdf và shattered-2.pdf. Mở các tệp này để kiểm tra sự khác biệt. Sau đó, tạo băm SHA-1 cho chúng và quan sát kết quả.
4. Rút ra kết luận dựa trên các quan sát, giải thích lý do cho sự tồn tại của xung đột trong MD5 và SHA-1.

II. Thực hiện ý tưởng:

1. So sánh hai thông điệp (Message 1 và Message 2):

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ nano msg1.hex
anhnqt@anhnqt-virtual-machine:~/Downloads$ cat msg1.hex
d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f895
5ad340609f4b30283e488832571415a085125e8f7cdc99fd91dbdf280373c5bd8
823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0e99
f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70
```

Lưu message 1 vào file msg1.hex

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ nano msg2.hex
anhnqt@anhnqt-virtual-machine:~/Downloads$ cat msg2.hex
d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f895
5ad340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5bd8
823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0e99
f33420f577ee8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70
```

Lưu message 2 vào file msg2.hex

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ cat msg1.hex | xxd -r -p > msg1.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$ cat msg2.hex | xxd -r -p > msg2.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$ cmp -l msg1.bin msg2.bin | wc -l
6
```

Chuyển 2 file hex string sang định dạng binary rồi so sánh từng byte khác biệt.

Nhận xét: Số byte khác biệt giữa hai thông điệp khá nhỏ có khoảng 6 byte khác nhau.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum msg1.bin
79054025255fb1a26e4bc422aef54eb4  msg1.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum msg2.bin
79054025255fb1a26e4bc422aef54eb4  msg2.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

kết quả MD5 của cả hai tệp msg1.bin và msg2.bin

Nhận xét: có vài byte khác biệt, kết quả MD5 của cả hai tệp msg1.bin và msg2.bin giống nhau hoàn toàn. Đây được gọi là cặp khối va chạm (collision blocks).

2. Hai chương trình thực thi hello và erase
    a. Chạy 2 file hello và erase

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ ./hello
Hello, world!

(press enter to quit)
anhnqt@anhnqt-virtual-machine:~/Downloads$ ./erase
This program is evil!!!
Erasing hard drive...1Gb...2Gb... just kidding!
Nothing was erased.

(press enter to quit)
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Kết quả sau khi thực thi 2 file hello và erase.

Nhận xét:

Chương trình hello in ra dòng chữ bình thường, trong khi chương trình erase giả lập việc xóa ổ cứng. Logic hoạt động hoàn toàn khác nhau.

b. Tính giá trị băm MD5:

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum hello
da5c61e1edc0f18337e46418e48c1290  hello
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum erase
da5c61e1edc0f18337e46418e48c1290  erase
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Kết quả sau khi tính giá trị băm MD5 cho 2 file hello và erase.

Nhận xét:

Kết quả mã băm MD5 của hai tệp này là trùng khớp. Hành vi thực thi hoàn toàn khác biệt một file in thông điệp bình thường, một file giả lập đe dọa xóa ổ cứng, mã băm MD5 của chúng lại giống hệt nhau. Kẻ tấn công có thể lợi dụng điều này để đánh tráo phần mềm an toàn bằng mã độc nhằm qua mặt các hệ thống kiểm tra tính toàn vẹn.

3. Phân tích file shattered-1.pdf và shattered-2.pdf

File shattered-1.pdf

File shattered-2.pdf

Nhận xét: màu sắc nền văn bản của 2 file khác nhau.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ sha1sum shattered-1.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-1.pdf
anhnqt@anhnqt-virtual-machine:~/Downloads$ sha1sum shattered-2.pdf
38762cf7f55934b34d179ae6a4c80cadccbb7f0a  shattered-2.pdf
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Kết quả sau khi băm SHA-1 với 2 file PDF trên.

Nhận xét:

- Kết quả trả về cho hai file là giống nhau hoàn toàn
- Nội dung hiển thị của hai file PDF khác nhau nhưng chúng chia sẻ chung một giá trị băm SHA-1.

4. Kết luận:

Kết luận: Dựa trên quan sát, tính chất "một chiều" (không thể dịch ngược) vẫn còn, nhưng tính "kháng va chạm" của MD5 và SHA-1 đã bị phá vỡ hoàn toàn.

Lý do: Các hàm băm này sử dụng cấu trúc khối Merkle-Damgård chia dữ liệu thành từng khối 64 bytes để xử lý. Lỗ hổng toán học trong thuật toán sinh khối cho phép kẻ tấn công đưa vào các "khối va chạm" (collision blocks). Những khối này bù trừ lỗi toán học cho nhau, khiến trạng thái nội tại (internal state) của hàm băm trở về y như cũ. Do đó, dù nội dung ở giữa khác nhau, giá trị băm ở cuối tệp vẫn giống nhau hoàn toàn.

Nhiệm vụ 2.3:

I. Ý tưởng: tạo ra hai tệp khác nhau có cùng giá trị băm MD5. Phần đầu của hai tệp này cần phải giống nhau, tức là chúng chia sẻ cùng một tiền tố (prefix). có thể đạt được điều này bằng cách sử dụng chương trình md5collgen, cho phép cung cấp một tệp tiền tố với nội dung tùy ý. Thực hành và trả lời các câu hỏi sau:

1. Nếu độ dài của tệp tiền tố không phải là bội số của 64, điều gì sẽ xảy ra?

2. Tạo một tệp tiền tố có chính xác 64 byte, chạy lại công cụ xung đột và xem điều gì xảy ra.

II. Thực hiện ý tưởng

1. Trường hợp tiền tố không phải là bội số của 64 bytes

tạo một file prefix.txt là file tiền tố ngẫu nhiên có độ dài không chia hết cho 64.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ nano prefix.txt
anhnqt@anhnqt-virtual-machine:~/Downloads$ cat prefix.txt
24520115
```

Tạo file prefix.txt với nội dung là mssv.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ ./tool_md5collgen -p prefix.txt -o out1.bin out2.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1.bin' and 'out2.bin'
Using prefixfile: 'prefix.txt'
Using initial value: 4e73152fcc008cf528e168f158a2a5de

Generating first block: ......
Generating second block: S00.................
Running time: 6.14964 s
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Chạy công cụ md5collgen để tạo file va chạm.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum out1.bin
d8ac3f5e49ab0ea1de745f9ffc8928a2  out1.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum out2.bin
d8ac3f5e49ab0ea1de745f9ffc8928a2  out2.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$ diff out1.bin out2.bin
Binary files out1.bin and out2.bin differ
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Kiểm tra mã băm MD5 và so sánh nội dung hai tệp đầu ra.

Nhận xét: Kết quả md5sum cho thấy hai tệp out1.bin và out2.bin có mã băm MD5 giống nhau. Lệnh diff lại thông báo hai tệp nhị phân này khác nhau (differ)

Nếu độ dài của file tiền tố không phải là bội số của 64:

- Giải thích: Cấu trúc thuật toán MD5 bắt buộc phải xử lý dữ liệu theo từng khối (block) có kích thước cố định là 64 byte.
- Quan sát thực tế: Nếu tệp prefix.txt không đủ 64 bytes (như ví dụ dài 8 bytes), công cụ md5collgen sẽ tự động chèn thêm các byte đệm (Padding) vào ngay sau tiền tố cho đến khi độ dài khối đạt vừa đủ 64 byte. Sau khi đã đệm đầy khối đầu tiên, nó mới gắn các khối va chạm (collision blocks) P và Q vào phía sau.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ xxd out1.bin
00000000: 3234 3532 3031 3135 0a00 0000 0000 0000  24520115........
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: e1f7 45ac ecd1 a9fe 1667 9091 5c1f 83fb  ..E......g..\...
00000050: cafe 871c 634b f6f2 4592 80ca 72ce a395  ....cK..E...r...
00000060: ffda fab9 5235 3b32 ebe9 c322 9bfb 0d85  ....R5;2..."....
00000070: fddb dd3a c8ef 146d df5f fd7a c742 1b76  ...:...m._.z.B.v
00000080: a218 2d6a 671c a512 bbba 64a9 d586 773f  ..-jg.....d...w?
00000090: de92 51df 19f5 c020 dd17 f3c8 5472 7db2  ..Q.... ....Tr}.
000000a0: 0189 57de 1b0a 9e82 c198 a0af 2ff9 526a  ..W........./.Rj
000000b0: 147e e13f 5e6b 8e80 6396 a16f e1f2 7ef6  .~.?^k..c..o..~.
anhnqt@anhnqt-virtual-machine:~/Downloads$ xxd out2.bin
00000000: 3234 3532 3031 3135 0a00 0000 0000 0000  24520115........
00000010: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: e1f7 45ac ecd1 a9fe 1667 9091 5c1f 83fb  ..E......g..\...
00000050: cafe 879c 634b f6f2 4592 80ca 72ce a395  ....cK..E...r...
00000060: ffda fab9 5235 3b32 ebe9 c322 9b7b 0e85  ....R5;2..."{..
00000070: fddb dd3a c8ef 146d df5f fdfa c742 1b76  ...:...m._...B.v
00000080: a218 2d6a 671c a512 bbba 64a9 d586 773f  ..-jg.....d...w?
00000090: de92 515f 19f5 c020 dd17 f3c8 5472 7db2  ..Q_... ....Tr}.
000000a0: 0189 57de 1b0a 9e82 c198 a0af 2f79 526a  ..W........./yRj
000000b0: 147e e13f 5e6b 8e80 6396 a1ef e1f2 7ef6  .~.?^k..c.....~.
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Mã HEX của 2 file output.

2. Trường hợp tiền tố dài chính xác 64 bytes

Tạo một file tiền tố prefix64.txt chứa chính xác 64 ký tự với việc lặp đi lặp lại 8 lần chuỗi 8 kí tự '24520115' (tương đương 64 bytes).

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ python3 -c "print('24520115' * 8, end='')" > prefix64.txt
anhnqt@anhnqt-virtual-machine:~/Downloads$ cat prefix64.txt
anhnqt@anhnqt-virtual-machine:~/Downloads$
```

Tạo file prefix_64.txt với 64 kí tự.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ ./tool_md5collgen -p prefix64.txt -o out1_64.bin out2_64.bin
MD5 collision generator v1.5
by Marc Stevens (http://www.win.tue.nl/hashclash/)

Using output filenames: 'out1_64.bin' and 'out2_64.bin'
Using prefixfile: 'prefix64.txt'
Using initial value: 67a661c53ffb41744a979eec46ca90bd

Generating first block: ....................................................
Generating second block: S10...................
Running time: 18.9414 s
```

Chạy công cụ md5collgen để tạo file va chạm.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ md5sum out1_64.bin out2_64.bin
7e9dcf2feed78190a23665d7d37caaa3  out1_64.bin
7e9dcf2feed78190a23665d7d37caaa3  out2_64.bin
anhnqt@anhnqt-virtual-machine:~/Downloads$ cmp out1_64.bin out2_64.bin
out1_64.bin out2_64.bin differ: byte 84, line 1
```

Kiểm tra mã băm MD5 và so sánh nội dung hai tệp đầu ra.

```
anhnqt@anhnqt-virtual-machine:~/Downloads$ xxd out1_64.bin
00000000: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000010: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000020: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000030: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000040: 9b9b 13dd 5ce3 b3a6 24df aaaf 4352 fdc0  ....\...$...CR..
00000050: bee5 f6e5 0f73 cb63 2d5e 80de 32aa 0b0b  .....s.c-^..2...
00000060: ad05 c2bd 4e23 ac85 eb6f 5d1a ddf3 2a42  ....N#...o]...*B
00000070: b747 2097 a263 bdb5 d48d c15e 7083 81d5  .G ..c.....^p...
00000080: 7be9 ebb0 db34 0db0 fd72 8d52 af41 cf47  {....4...r.R.A.G
00000090: 3376 0353 68dc a108 abb5 3037 1b60 a8b7  3v.Sh.....07.`..
000000a0: cbf5 4418 9702 af7d f449 449b 9cd7 029f  ..D....}.ID.....
000000b0: f630 8244 9287 4230 0cda 1ed3 f911 d730  .0.D..B0.......0
anhnqt@anhnqt-virtual-machine:~/Downloads$ xxd out2_64.bin
00000000: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000010: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000020: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000030: 3234 3532 3031 3135 3234 3532 3031 3135  2452011524520115
00000040: 9b9b 13dd 5ce3 b3a6 24df aaaf 4352 fdc0  ....\...$...CR..
00000050: bee5 f665 0f73 cb63 2d5e 80de 32aa 0b0b  ...e.s.c-^..2...
00000060: ad05 c2bd 4e23 ac85 eb6f 5d1a dd73 2b42  ....N#...o]..s+B
00000070: b747 2097 a263 bdb5 d48d c1de 7083 81d5  .G ..c......p...
00000080: 7be9 ebb0 db34 0db0 fd72 8d52 af41 cf47  {....4...r.R.A.G
00000090: 3376 03d3 68dc a108 abb5 3037 1b60 a8b7  3v..h.....07.`..
000000a0: cbf5 4418 9702 af7d f449 449b 9c57 029f  ..D....}.ID..W..
000000b0: f630 8244 9287 4230 0cda 1e53 f911 d730  .0.D..B0...S...0
```

Mã HEX của 2 file output.

Nhận xét: Khi file tiền tố có chính xác 64 bytes đã lấp hoàn toàn một khối xử lý của thuật toán MD5. Do đó, công cụ md5collgen không chèn thêm bất kỳ byte đệm (padding) nào. Nó trực tiếp nối các khối dữ liệu va chạm (collision blocks) ngay sau khối tiền tố này. Điều này chứng minh rằng việc tạo va chạm trong MD5 luôn được thực hiện tại ranh giới của các khối 64 bytes.